

Task 3(A) Array Manipulation And Analysis Programs.

Return The Max sliding Window

Aim: To write a Java program to find the maximum value in each sliding window of size k as the window moves from left to right across a given integer array.

Algorithm:

- (1) Start
- (2) Read the array nums and window size k
- (3) Create an output array of size $n - k + 1$
- (4) for each window starting from index $i = 0$ to $n - k$:
 - * Assume the first element of the window is the maximum
 - * Compare all k elements in the current window
 - * update the maximum value
 - * store the maximum in the result array
- (5) Print the result.

Program:

```
import java.util.Scanner;
class SlidingWindowMaximum {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter array size:");
        int n = sc.nextInt();
        int[] nums = new int[n];
        System.out.println("Enter " + n + " array elements:");
        for (int i = 0; i < n; i++) {
            nums[i] = sc.nextInt();
        }
        System.out.print("Enter window size:");
        int k = sc.nextInt();
        int[] result = new int[n - k + 1];
        for (int i = 0; i < n - k + 1; i++) {
            int max = nums[i];
            for (int j = i + 1; j < i + k; j++) {
                if (nums[j] > max) {
                    max = nums[j];
                }
            }
            result[i] = max;
        }
    }
}
```



```
System.out.print("Sliding window maximum:");  
for (int i=0; i< result.length; i++) {  
    System.out.print(result[i] + " ");  
}  
}
```


Output

Enter array size : 8

Enter 8 array elements:

1 3 -1 -3 5 3 6 7

Enter window maximum: 3 3 5 5 6 7

TASK : 3(B)

Adjusted Sum

Aim: To write a Java program that calculates the adjusted sum of an integer array by adding all even numbers and subtracting all odd numbers.

Algorithm:

- (1) start
- (2) Read the value of N
- (3) Read N integer elements into the array
- (4) Initialize a variable sum to 0.
- (5) Traverse each element of the array:
 - * if the element is even, add it to sum.
 - * if the element is odd, subtract it from sum.
- (6) Display the final adjusted sum.
- (7) stop

Program

```
import java.util.Scanner;
class AdjustedSum {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the size of the array:");
        int N = sc.nextInt();
        int[] arr = new int[N];
        int sum = 0;
        System.out.println("Enter the array elements:");
        for (int i = 0; i < N; i++) {
            arr[i] = sc.nextInt();
            if (arr[i] % 2 == 0) {
                sum = sum + arr[i];
            } else {
                sum = sum - arr[i];
            }
        }
        System.out.println("Adjusted Sum = " + sum);
    }
}
```

VEL TECH - CSE	
EX NO.	
PERFORMANCE (5)	
RESULT AND ANALYSIS (5)	
DATE (5)	

Output

Enter the size of the array : 5

Enter the array elements :

10 15 20 25 30

Adjusted Sum = 20

Task-3(C) Rotate array and find Max Difference

Aim: To write a java program that rotates an array to the right by k positions and finds the maximum absolute difference between any two adjacent elements after rotation.

Algorithm:

- (1) Start
- (2) Read the size of the array N.
3. Read N elements into the array.
4. Read the value of k.
5. Rotate the array to the right by k positions using the formula:
$$\text{rotated}[(i+k) \% N] = \text{arr}[i]$$
6. Traverse the rotated array and calculate the absolute difference between adjacent elements using `Math.abs()`.
7. Store the maximum difference found.
8. Display the rotated array and the maximum difference.
9. Stop.

Program:

```
import java.util.Scanner;
class RotateMaxDifference {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the size of the array: ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        int[] rotated = new int[n];
        System.out.println("Enter the array elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        System.out.print("Enter k value: ");
        int k = sc.nextInt();
        k = k % n;
        for (int i = 0; i < n; i++) {
            rotated[(i+k) % n] = arr[i];
        }
        int maxDiff = 0;
        for (int i = 0; i < n-1; i++) {
            int diff = Math.abs(rotated[i] - rotated[i+1]);
            if (diff > maxDiff) {
                maxDiff = diff;
            }
        }
        System.out.println("Rotated Array:");
        for (int num : rotated) {
            System.out.print(num + " ");
        }
    }
}
```


Output

Enter the size of the array : 5

Enter the array elements :

3 8 9 7 6

Enter k value : 1

Rotate Array :

6 3 8 9 7

Maximum difference = 5


```

    }
    System.out.println("Maximum difference = " + maxDiff);
}
}

```

Input:
 Vehicle ID: 101
 Model Name: Honda City
 Base Rent: 2000

Output:
 Vehicle ID: 101
 Model Name: Honda City
 Base Rent: 2000

Input:
 Vehicle ID: 101
 Model Name: Honda City
 Base Rent: 2000.0
 Total Rent: 2200.0

Output:
 Vehicle ID: 101
 Model Name: Honda City
 Base Rent: 2000.0
 Total Rent: 2200.0

VEL TECH - CSE	
EX NO.	3
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	5
RECORD (5)	5
TOTAL (20)	20
SIGN WITH DATE	<i>[Signature]</i> 10/02/20

Result: Thus, the java program successfully rotates the array to the right by k positions and finds the maximum absolute difference between adjacent elements.